



Planilla de Corrección Proyecto 1
Comisión 21

Nota final: C-

Aprobado condicionado a reentregar proyecto con errores resueltos.
Tomar contacto con el ayudante a cargo para debatirlos errores.

Análisis preliminar del proyecto: *descarga y compilación del código fuente; uso de la aplicación en ejecución.*

¿Todos los alumnos mostraron actividad en el uso del repositorio y las defensas?

SI

- Se observan commits de consideración de todos los alumnos, a excepción del alumno CONTI, Iván que abandonó la materia.

¿El código se compila?

SI

- El código se compila correctamente, y ante la ejecución no se visualizan errores por consola.

¿El juego muestra tiempo, puntaje y vidas en todo momento? ¿El juego implementa dos modos de juego diferentes? ¿El juego opera el ranking de forma consistente?

SI

- El juego muestra puntaje, monedas, tiempo restante, vidas y nivel actual en todo momento.
- El juego implementa dos modos de juego diferentes aunque algunos sprites son los mismos que el juego original.
- El ranking se muestra correctamente desde la pantalla de inicio y al finalizar la partida, ya sea por victoria o derrota.

¿El juego se comporta como se espera (estado inicial, condición de finalización, transición de como mínimo 3 niveles)?

SI

- Estado Inicial: al comenzar la partida las imágenes de los elementos del juego son mostrados correctamente.
- Condición de finalización: tanto al perder todas las vidas como al llegar al final del juego la partida finaliza correctamente.
- Transiciones: se implementa la transición de tres niveles.

¿El juego se comporta como se espera ante la interacción con los power-ups (cambio de estado, cambio de sprites)?

SI

- Al interactuar con el SuperChampiñon no se aumenta el tamaño del personaje principal. El resto de los PowerUps funciona correctamente.
- Los bloques de pregunta no cambian su imagen para indicar que quedan desactivados luego de la primera interacción.

¿El juego se comporta como se espera ante la interacción con los enemigos (condición para muerte, estados intermedios de enemigos tales como tortuga)?

SI

- La PiranhaPlant sale y se oculta muy rápido, por lo que no se puede probar si es inmune cuando está oculta.
- El resto de enemigos se comportan adecuadamente.
- Cuando el jugador está en estado SuperMario y colisiona con un enemigo este muere inmediatamente. Se debería dar un período de gracia para poder visualizar la transición al EstadoNormal.

¿El juego se comporta como se

NO



Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
Tecnología de Programación (TdP)
Segundo Cuatrimestre 2024



espera respecto del lanzamiento de bolas de fuego?	- Las bolas de fuego no pueden ser lanzadas hacia atrás. - Las bolas de fuego deberían desaparecer al colisionar con Plataformas. - Las interacciones con Enemigos son correctas.
Virtudes de la GUI y usabilidad	- Sin observaciones.
Debilidades de la GUI y usabilidad	SI - Las entidades no cuentan con animaciones. - Mario no mira en la dirección hacia la que camina y no puede lanzar bolas de fuego hacia atrás. - Algunos sprites del modo de juego alternativo son los mismos que se utilizan en el modo original. - Si Mario cae de una Plataforma y el jugador presiona para saltar, Mario salta desde el aire en lugar de seguir cayendo.
Errores y/o características que deberían modificarse	SI - Cuando el personaje principal colisiona con los enemigos desde arriba y no está bien centrado (especialmente cuando se está en estado SuperMario) el jugador recibe daño. Se debe revisar la detección de colisiones.

(sigue debajo)



Análisis preliminar del proyecto: *diagramas reducidos, extendidos y de secuencia.*

¿Se encuentran accesibles los *diagramas reducidos y extendidos* de clases de cada una de las defensas?

SI

¿Se encuentran accesibles los *diagramas de secuencia* solicitados para la defensa correspondiente?

SI

- Sin observaciones.

¿Se encuentran accesibles los *diagramas reducido y completo de clases* de la entrega final?

SI

- Observaciones sobre el diagrama:
- Colocar en las asociaciones el nombre de la relación y su cardinalidad (1, 0..*, 1..*, n..m, etc). Esto da mayor entendimiento sobre la relación entre las clases.
- Hay atributos en clases del diagrama extendido que no están mostradas como asociaciones en el diagrama reducido. Por ejemplo, HiloEnemigo con ControladorColisiones.

- Observaciones sobre el modelado:
(Diagrama reducido)

- Sobre el manejo de sonidos. Cómo la emisión de sonidos es producida desde múltiples clases, resulta que cada clase conozca los sonidos que puede emitir y que se utilice el patrón Singleton para activar la reproducción de sonido cuando lo necesite.

(Diagrama extendido)

- ¿Por qué modelar el atributo PowerUp de BloqueDePregunta con un string? ¿Por qué no respetar la asociación planteada en el diagrama reducido? Esta decisión no es extensible. (Más detalles en la sección de análisis de código).

- Hay métodos que no están en el código que sí lo están en el diagrama tales como recibirPuntosEstrella(). Si no se encuentran en el código se los debe eliminar para mantener la consistencia.

- El método esInvulnerable() en los estados no parece apropiado teniendo modelado el Estado Invulnerable. Se debe aprovechar este estado para que cuando Mario deba recibir daño, éste lo delegue a su Estado actual. Algo similar ocurre con esGrande() y SuperMario.

- Elemento tiene atributos para coordenadas "x" e "y" y también un atributo Rectangle para hitbox el cual también se usa para la posición. Los atributos posicionX, posicionY podrían eliminarse y utilizar la posición x e y del Rectangle.

- Hay atributos que están modelados como públicos. Por ejemplo, en la clase Jugador. Esto no es correcto considerando el encapsulamiento.

(sigue debajo)



Análisis del código fuente: *inspección del código fuente entregado.*

¿El código fuente se encuentra adecuadamente organizado en paquetes?

SI

- Para mejorar la organización se podría usar una estructura de paquetes jerárquica. Por ejemplo, que los paquetes enemigos, plataformas y powerups estén dentro del paquete elementos.

¿El código exhibe de forma consistente y adecuada las prácticas propuestas por Clean Code?

SI

- En general se observan prácticas de clean code a lo largo del código, se utilizan nombres significativos y métodos pequeños. Salvo en escasos casos como los siguientes.
- Se observan métodos largos que podrían refactorizarse en métodos privados más pequeños. Tal es el caso del método `run()` de `HiloEnemigo`.
- Ocasionalmente se observan "magic numbers" que deberían ser colocados en constantes para entender qué significan. Por ejemplo, el 7471 dentro del método `actualizar()` de `Jugador`.
- Hay métodos que tienen código con distinto nivel de abstracción lo cual reduce su legibilidad. Por ejemplo, el método `actualizar()` de `Enemigo`.

¿Existe una adecuada división de responsabilidades entre las clases que implementan cuestiones de lógica/gráfica?

NO.

- Se utiliza el patrón Observer correctamente para mostrar las imágenes de los elementos del juego. Sin embargo, al momento de eliminar un observer, se hace una cadena de llamados desde `Nivel`, Pasando por `ControladorPartida` y `ControladorPantalla` hasta `PantallaJuego`. En un momento de esta cadena de llamados se hace un casteo de `Observer` a `ObserverGrafico`. Para evitar esto, lo que se debería hacer es que la interfaz `observer` ofrezca un método para eliminarse y que `ObserverGrafico` tenga una asociación con la clase `ControladorPantalla`. Esto último tiene sentido porque ambas forman parte de la capa vista. Entonces, cuando un `Elemento` deba ser eliminado, este le comunica a su `Observer` que también tiene que ser eliminado.

- Cuando los elementos del juego son creados se asigna su hitbox según el tamaño que tiene la imagen. Esto no es lo ideal ya que se pierde el control dentro del código del tamaño de los Elementos del juego al ser dependiente del archivo, lo cual puede traer problemas si este último es modificado. Una mejor opción es definir el tamaño de las hitbox de los Elementos por medio de constantes y cuando los observers tengan que mostrar las imágenes que éstos las escalen al área definida .

- Métodos `getter` que devuelven atributos internos tales como `getHiloEnemigos()` de `ControladorPartida` o `getState()` en `Jugador` rompen de cierta forma el encapsulamiento ya que las clases clientes quedan acopladas a los tipos de los atributos internos. ¿Qué sucede si el `ControladorPartida` deja de tener un hilo para enemigos y pasara a tener un hilo para todo? Tendríamos que cambiar todos los lugares del código en los que se llame a `getHiloEnemigos()`. Más apropiado sería que, si se quiere agregar un enemigo, que `ControladorPartida` provea un método para ello y que luego, internamente, lo agregue a su hilo si corresponde. De esta forma, las clases



Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
Tecnología de Programación (TdP)
Segundo Cuatrimestre 2024



	clientes se abstraen de cómo funciona ControladorPartida internamente (encapsulamiento). - La clase Nivel no parece tener ninguna responsabilidad más allá de mantener las colecciones de los distintos tipos de elementos, algunas de las cuales también se encuentran repartidas en las clases HiloEnemgo. Parece más apropiado que las colecciones estén en ControladorColisiones que es la clase que las utiliza a todas para realizar sus operaciones. - Los métodos que crean Elementos durante el juego tales como lanzarSpiny() o lanzarBolaDeFuego() realizan operaciones tales como registrar los elementos al hilo correspondiente y su observer. Esto sería más bien una responsabilidad de la fábrica cuando crea los Elementos.
¿Se implementa de forma consistente la <u>detección</u> de colisiones?	NO. - La solución propuesta de subdividir el área principal en 4 subáreas pequeñas es simple en teoría pero genera complicaciones en la jugabilidad. Las áreas laterales y el tamaño reducido del área inferior provocan que el jugador se muera ante un enemigo si cae encima con su borde.
¿Se implementa de forma consistente la <u>resolución</u> de colisiones?	NO - Se implementa el patrón Visitor con una relación “es-un”, lo cual es aceptable, pero se utilizan interfaces implementadas sólo por aquellos elementos que pueden interactuar entre sí. Esto parece complejizar de más el patrón, ¿por qué no tener un solo Visitor con los métodos visit(Jugador), visit(Enemigo), visit(Plataforma) y visit(PowerUp), y Visitable un único método accept(Visitor) como lo establece el patrón?
¿Se implementa de forma consistente los diferentes estados de Mario?	NO. - Los métodos esInvulnerable() y esGrande() no deberían estar habiendo modelado los Estados que representan estos conceptos. - En lugar de que todo aquello que colisione con el Jugador decida qué hacer según el Estado del mismo, debería ser Jugador quién decida qué hacer según su Estado. De esta forma, las clases clientes a Jugador no quedan acopladas a los Estados. Por ejemplo, en el método visitar(Jugador) de Buzzy, en lugar de preguntar si el Jugador es Invulnerable(), directamente llamar a un método de Jugador que se llame por ejemplo recibirDaño(Enemigo). Dentro de este método la clase Jugador delegará en su Estado actual, llamando por ejemplo recibirDaño(Enemigo), la decisión de qué acción realizar (morir, seguir vivo, etc)
Virtudes	SI - El patrón factory es implementado de manera super extensible: lo único que habría que hacer es crear un tipo de fábrica nueva y pasarle por constructor la ruta de la carpeta con los archivos nombrados apropiadamente.
Debilidades	SI - Observaciones:



	- Se debe utilizar el .gitignore para que los archivos que no sean código fuente sean ignorados (.project, /bin, .settings, .classpath, etc). Es decir que no sean trackeados por Git y que en consecuencia no se suban a GitHub. - La creación de niveles no es fácil de extender. En el ControladorPartida se consulta si el número de nivel es menor a 3 y NivelBuilder en su constructor recibe un número con el cual se decide qué nivel crear. ¿En cuántos lugares se tiene que modificar el código si quiero agregar un nuevo nivel? Por lo menos en las dos clases mencionadas anteriormente. Una alternativa más adecuada sería que NivelBuilder reciba el archivo que va a construir y tenga una referencia a un siguiente NivelBuilder. La clase encargada de transicionar de nivel solo tendría que consultar si su NivelBuilder actual tiene un siguiente. En caso de no tenerlo, finaliza la partida. De esta forma, cuando se quiera agregar o sacar un nivel, simplemente se modifica el código que se encarga de crear los NivelBuilder y establecer sus siguientes. - En HiloEnemigo. Si se va a iterar sobre un tipo List, es más conveniente utilizar la estructura de datos LinkedList en lugar de ArrayList (a menos que se necesite indexar) y usar foreach que es más legible que iterator. - Utilizar un string para indicar el PoweUp que tiene el BloqueDePregunta para decidir cual aparece es muy poco extensible ya que si se quisiera agregar un powerUp tendría que modificar el código de BloqueDePregunta. La única responsabilidad del BloqueDePregunta es almacenar un PowerUp (sin saber qué tipo específico es) y hacerlo aparecer cuando corresponda. Para esto, PowerUp debería tener un atributo PowerUp (como está modelado en el diagrama reducido) y lo recibe como parámetro en el constructor. - La clase Launcher debería ser responsable solo de armar la aplicación y ejecutarla, es decir, crear ControladorPartida y ControladorPantallas. La creación del Ranking debería manejarse dentro de otra clase.
Errores y/o características que deberían modificarse	SI - Cambiar la utilización del patrón observer al momento de eliminar las imágenes. La interfaz Observer debería proveer un método eliminar(), la clase ObserverGráfico debería tener una asociación con la pantalla que los muestra. Entonces, cuando una Entidad recibe el mensaje de que debe eliminarse, ésta le indica a su observer que también se elimine. Se deben delegar las responsabilidades en lugar de usar cadena de getters para llegar a un atributo interno de otra clase para realizar la acción. Por ejemplo, cuando la BolaDeFuego toca a un Enemigo, el Enemigo debería proveer un método morir() y se debería llamar a este método. No tiene sentido que la BolaDeFuego se encargue de eliminar el observer del Enemigo que toca. - Lineas tales como "this.nivel.controladorPartida.controladorSonido.reproducirSonidoAccion(TipoSonidos.muerteEnemigo);" en el método saltarAlMatar() de Jugador rompen el encapsulamiento ya que se está accediendo a los atributos internos directamente. Cuando se hace esto, o cadenas de getters que devuelven



Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
Tecnología de Programación (TdP)
Segundo Cuatrimestre 2024



atributos internos, se debe revisar el modelo porque es señal de que las responsabilidades y asociaciones entre clases no son adecuadas.

- Utilizar patrón Singleton para el manejo de sonidos. Esto soluciona la cadena de getters en el punto anterior.
- Revisar las responsabilidades de las clases que ofrecen gets de atributos internos y cambiar estos métodos por otros que provean servicios a las clases clientes preservando el encapsulamiento.